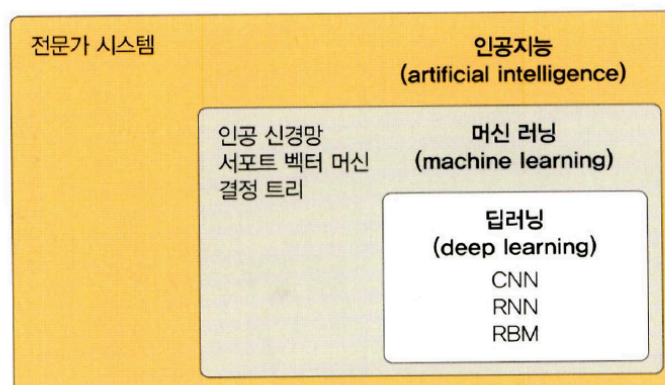


1주차 예습과제-딥러닝 파이토치 교과서 1~2장

1장-머신 러닝과 딥러닝

1.1 인공지능, 머신 러닝과 딥러닝

▼ 그림 1-1 인공지능과 머신 러닝, 딥러닝의 관계



- **인공지능:** 인간의 지능을 모방하여 사람이 하는 일을 컴퓨터(기계)가 할 수 있도록 하는 기술
 - 인공지능을 구현하는 방법으로는 머신러닝, 딥러닝이 있음
 - 두 방법 모두 학습 모델을 제공하여 데이터를 분류하는 기술이지만 명확한 차이가 존재!
- **머신러닝**
 - 데이터 전처리 과정 필요
 - 데이터의 특징을 스스로 추출 X
 - 각 데이터의 특성을 컴퓨터에 인식시키고 학습시켜 문제를 해결
- **딥러닝**
 - 인간이 하던 작업을 생략
 - 대량의 데이터를 신경망에 적용하면 컴퓨터가 스스로 분석 후 답을 찾음

▼ 표 1-1 머신 러닝과 딥러닝

구분	머신 러닝	딥러닝
동작 원리	입력 데이터에 알고리즘을 적용하여 예측을 수행한다.	정보를 전달하는 신경망을 사용하여 데이터 특징 및 관계를 해석한다.
재사용	입력 데이터를 분석하기 위해 다양한 알고리즘을 사용하며, 동일한 유형의 데이터 분석을 위한 재사용은 불가능하다.	구현된 알고리즘은 동일한 유형의 데이터를 분석하는 데 재사용된다.
데이터	일반적으로 수천 개의 데이터가 필요하다.	수백만 개 이상의 데이터가 필요하다.
훈련 시간	단시간	장시간
결과	일반적으로 점수 또는 분류 등 숫자 값	출력은 점수, 텍스트, 소리 등 어떤 것이든 가능

1.2 머신 러닝이란

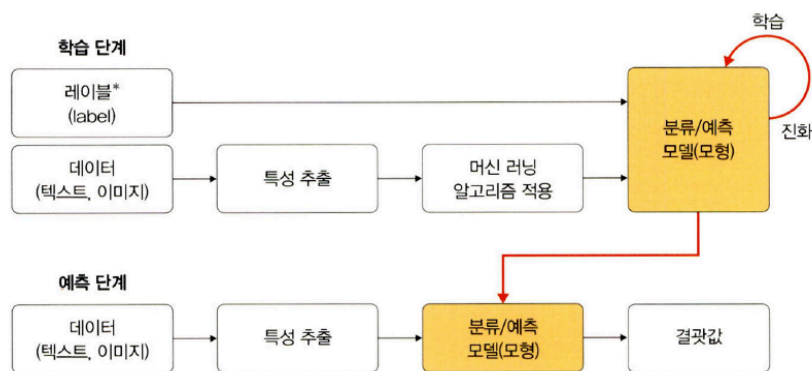


머신 러닝은 인공지능의 한 분야로, 컴퓨터가 학습할 수 있게 하는 알고리즘과 기술을 개발하는 분야

1.2.1 머신 러닝 학습 과정

- 학습 단계(learning)와 예측 단계(prediction)으로 구분
 - 학습 단계: 훈련 데이터를 머신 러닝 알고리즘에 적용 → 학습 결과로 모형 생성
 - 예측 단계: 학습 단계에서 생성된 모형에 '새로운 데이터'를 적용하여 결과 예측

▼ 그림 1-3 머신 러닝 학습 과정



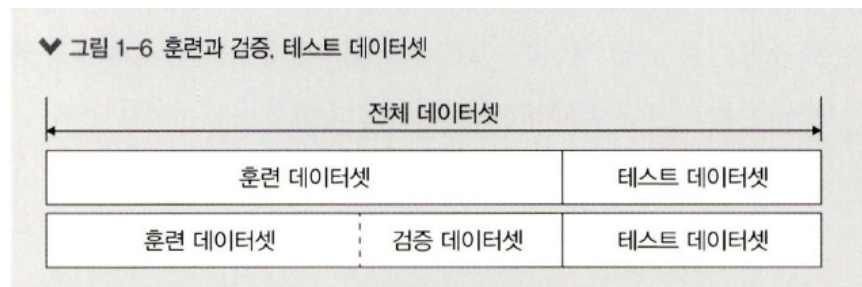
* 레이블은 지도 학습에서 정답을 의미

☀특성 추출: 데이터별로 어떤 특징을 가지고 있는지 찾아내고, 그것을 토대로 데이터를 벡터로 변환하는 작업 (사람이 인지하는 데이터를 컴퓨터가 인지할 수 있는 데이터로 변환해주는 작업 필요!!)

- 머신 러닝의 주요 구성 요소: 데이터&모델

- 데이터

- 머신 러닝이 학습 모델을 만드는 데 사용하는 것
- 실제 데이터의 특징이 잘 반영되고 편향되지 않은 훈련 데이터를 확보하는 것이 중요
- 용도에 따라 '훈련 데이터셋(80%)' 과 '테스트 데이터셋(20%)'으로 분리 (or '훈련 데이터셋'을 또다시 '훈련 데이터셋' 과 '검증 데이터셋'으로 분리)

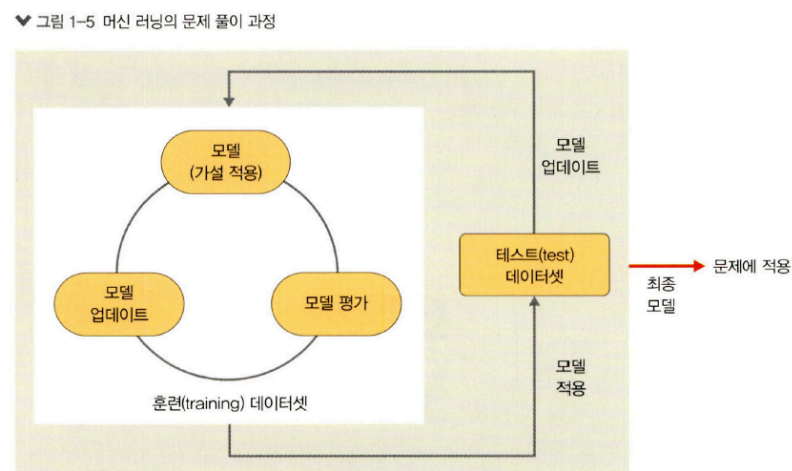


- 모델

- 머신 러닝의 학습 단계에서 얻은 최종 결과물(가설)
- 모델의 학습 절차:

- 모델 선택
- 모델 학습 및 평가
- 평가를 바탕으로 모델 업데이트

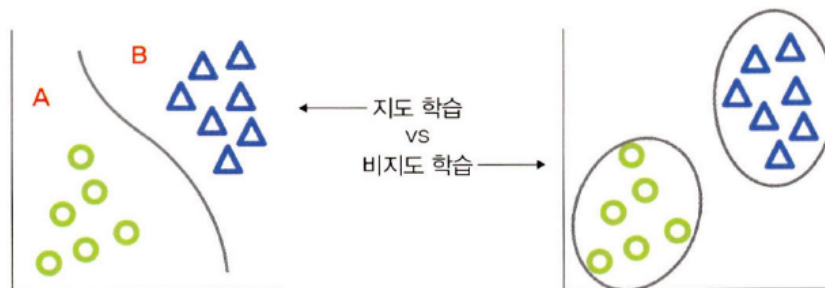
이 3단계를 반복하면서 최적의 모델 찾기



1.2.2 머신 러닝 학습 알고리즘

- **지도 학습**: 정답이 무엇인지 컴퓨터에 알려 주고 학습시키는 방법
- **비지도 학습**: 정답을 알려 주지 않고 특징이 비슷한 데이터를 클러스터링(범주화)하여 예측하는 학습 방법

▼ 그림 1-8 지도 학습과 비지도 학습



- **★강화학습**: 분류할 데이터가 있더라도 '정답'이 정해져 있지 않음.
자신의 행동에 대한 보상을 받으며 학습을 진행!!
보상이 커지는 행동은 더 자주 하도록 하고, 줄어드는 행동은 덜 하도록 하며 학습함

▼ 표 1-2 지도 학습, 비지도 학습, 강화 학습

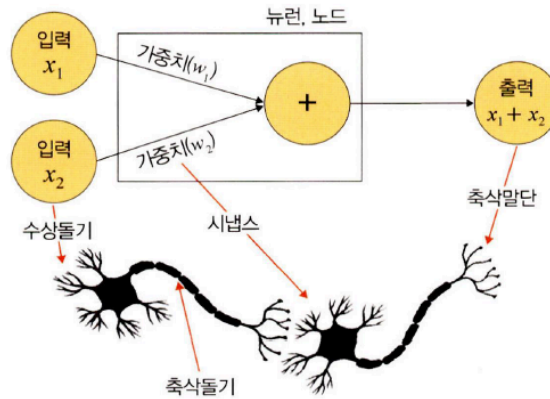
구분	유형	알고리즘
지도 학습 (supervised learning)	분류(classification)	• K-최근접 이웃(K-Nearest Neighbor, KNN) • 서포트 벡터 머신(Support Vector Machine, SVM) • 결정 트리(decision tree) • 로지스틱 회귀(logistic regression)
	회귀(regression)	선형 회귀(linear regression)
비지도 학습 (unsupervised learning)	군집(clustering)	• K-평균 군집화(K-means clustering) • 밀도 기반 군집 분석(DBSCAN)
	차원 축소 (dimensionality reduction)	주성분 분석 (Principal Component Analysis, PCA)
강화 학습 (reinforcement learning)	-	마르코프 결정 과정 (Markov Decision Process, MDP)

1.3 딥러닝이란

- **🧠 딥러닝**: 인간의 신경망 원리를 모방한 심층 신경망 이론을 기반으로 고안된 머신 러닝 방법의 일종 (인간의 뇌를 기초로 하여 설계)
 - 인간의 뇌가 뉴런과 시냅스로 구성되어 있는 것에 착안

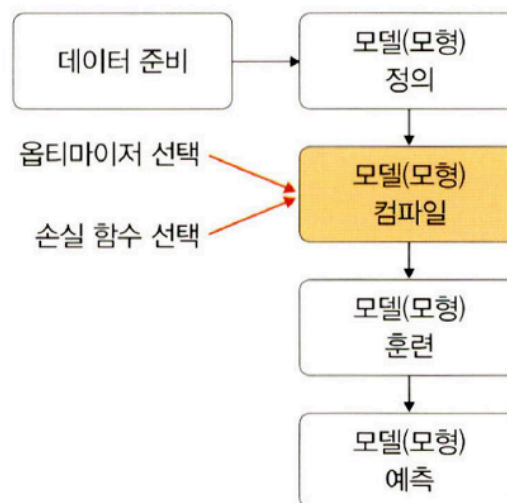
- 각각의 뉴런은 복잡하게 연결된 수많은 뉴런을 병렬 연산하여 음성, 영상 인식 등을 처리

▼ 그림 1-10 인간의 신경망 원리를 모방한 심층 신경망



1.3.1 딥러닝 학습 과정

▼ 그림 1-11 딥러닝 모델의 학습 과정



1. 데이터 준비

- 초보자가 데이터 구하는 방법 2가지
 - 프레임워크 내장 데이터셋: 파이토치(tutorials.pytorch.kr), 케라스(keras.io)
 - 장점: 이미 전처리돼 있어 바로 사용 가능, 예제 코드가 많음
 - 공개데이터: 캐글(kaggle.com)-AI 경진대회 플랫폼, 공개 데이터 많음
- 국내 공개 데이터도 가능하지만 전처리가 많이 필요 → 처음엔 캐글 같은 플랫폼 권장

2. 모델(모형) 정의

- 이 단계에서 신경망을 생성
- 은닉층 개수 ↑ → 성능 ↑, 그러나 과적합 확률 ↑ → 둘은 상충 관계(트레이드오프)
- 과적합(overfitting): 훈련 데이터를 과하게 학습해 훈련 데이터 오차는 줄지만 새 데이터 오차는 커지는 현상
- 그래서 정의 단계에서 신경망을 "제대로" 만드는 게 중요

3. 모델(모형) 컴파일

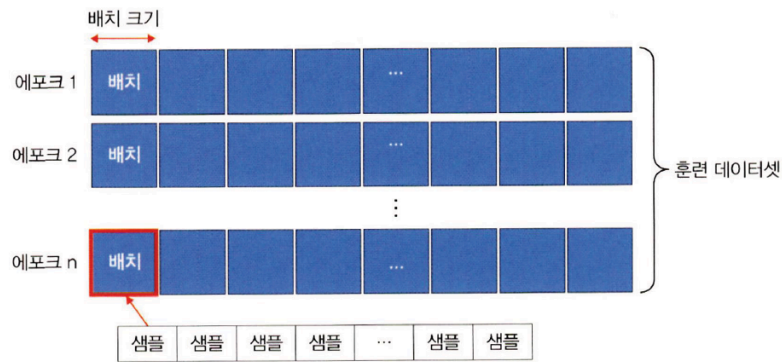
- 여기서 3가지를 선택: 활성화 함수 / 손실 함수 / 옵티마이저
 - 활성화 함수: 입력 신호가 기준 이상이면 출력 신호로 바꿔주는 함수. 시그모이드, 하이퍼볼릭 탄젠트, ReLU 등
 - 손실함수: 모델 출력값과 원하는 출력값(레이블)의 차이 = 오차를 구하는 함수 (MSE, Cross Entropy)
 - 옵티마이저: 손실 함수 값을 기반으로 네트워크를 어떻게 업데이트할지 정하는 방법 (Adam, RMSProp 등)
- 데이터 형태에 따라 선택이 달라짐

훈련 데이터 형태	손실함수
연속형 (숫자 예측)	평균 제곱 오차 MSE
이진 분류(둘 중 하나)	크로스 엔트로피

4. 모델(모형) 훈련

- 한 번에 처리할 데이터양을 지정하는 단계
- 한 번에 너무 많이 처리하면 → 학습 속도 느려짐 + 메모리 부족
- 그래서 전체 데이터를 일정 묶음으로 나눔 → 배치, 훈련 반복 횟수 → 에포크
- 훈련 중 값의 변화를 시각화하며 파라미터·하이퍼파라미터 최적값을 찾아야 함
 - 파라미터: 모델이 학습으로 스스로 찾는 값 (가중치 등)
 - 하이퍼파라미터: 사람이 정해주는 값 (배치 크기, 에포크, 학습률 등)

▼ 그림 1-12 모델 훈련에 필요한 하이퍼파라미터



배치, 에포크 계산

- 데이터 1000개, 배치 크기 20 → 샘플 20개마다 가중치 2회 업데이트 → 1 에포크에 50번
- 에포크 10 → 50번×10=총 500번 업데이트
- 각 샘플은 에포크마다 한 번씩 → 총 10번 사용됨
- 공식: 총 업데이트=(데이터 수 ÷ 배치 크기) × 에포크

▼ “성능이 좋다” 는 의미는?

- ML/DL에서 성능 (performance)의 공식 정의는 없음
- 데이터가 수집된 산업 분야, 모델 목적에 따라 달라짐
- 보통 다음 중 하나를 뜻함: 예측을 잘한다(정확도 높음)/ 훈련 속도가 빠르다

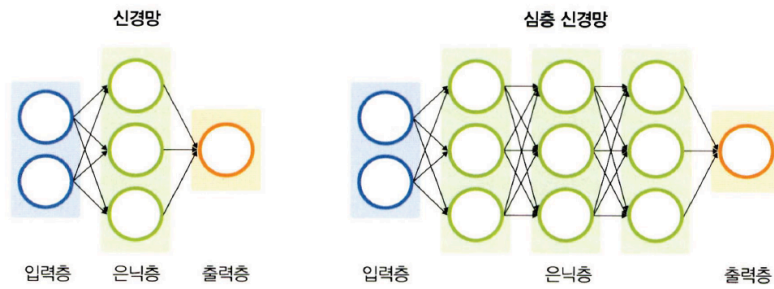
5. 모델(모형) 예측

- 검증 데이터셋을 만든 모델에 넣어 실제 예측을 해보는 단계
- 예측력이 낮으면 → 파라미터 튜닝 또는 신경망 자체를 재설계

핵심 구성 요소: 신경망과 역전파

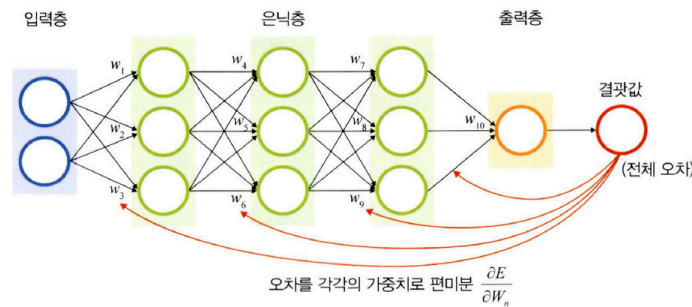
- 딥러닝= 머신러닝의 한 분야지만, 심층 신경망 (DNN)을 쓴다는 점이 다름
 - 심층 신경망: 은닉층이 2개 이상인 신경망
 - 데이터셋의 어떤 특성이 중요한지 스스로 알아내는 기능 → ML은 사람이 특징을 뽑아줘야 하는데 DL은 이걸 자동으로

▼ 그림 1-13 신경망과 심층 신경망



- **역전파(backpropagation):** 가중치 값을 업데이트하기 위한 계산
 - 출력층에서 나온 전체 오차 E 를 각 가중치 W_n 로 편미분 $\rightarrow \partial E / \partial W_n$
 - 이 미분 계산이 성능에 영향을 주는 주요 요소
- 파이토치 같은 프레임워크가 역전파를 자동 처리 $\rightarrow$ 딥러닝 구현이 훨씬 간단해짐

▼ 그림 1-14 역전파 계산



1.3.2 딥러닝 학습 알고리즘

1. 지도 학습 (supervised)

- 이미지 분류, 인식, 분할 $\rightarrow$ CNN (합성곱 신경망)
 - 분류: 이미지가 어떤 클래스에 속하는지 $\rightarrow$ 유사한 것끼리 묶을 때
 - 인식: 사진 속 사물의 종류 식별 $\rightarrow$ 의료 이미지 질병 식별, 산업 검사, 로봇 비전
 - 분할: 객체 간 영역을 픽셀 단위로 구분 $\rightarrow$ X-ray, CT, MRI 등 의료 영상 진단
- 시계열 데이터 $\rightarrow$ RNN
 - 시간에 따른 데이터(주식 등)에 사용
 - 단점: 역전파 과정에서 기울기 소멸
 - 해결: LSTM = RMM + 게이트 3개

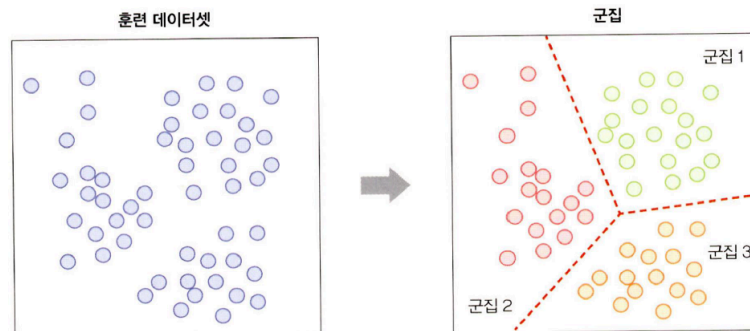
- 망각 게이트(과거 정보 잊기) / 입력 세이트(현재 정보 기억) / 출력 게이트 (최종 결과)

- 결과: 현재 시계열 문제에서 가장 활발히 사용

2. 비지도 학습 (unsupervised)

- 워드 임베딩
 - 자연어를 컴퓨터가 처리하도록 단어를 벡터로 표현
 - 대표: Word2Vec, GloVe
 - 활용: 번역, 음성 인식 (자연어 처리 분야)
- 군집 (clustering)
 - 정보 없는 상태에서 데이터 분류, 클러스터 내부는 비슷하게, 클러스터끼리는 구분 되게
 - 머신 러닝의 군집과 같음. 단, 딥러닝과 함께 쓰면 성능 ↑ (신경망 안에서 군집 알고리즘 사용)

▼ 그림 1~18 군집



3. 전이 학습 (transfer learning)

- 사전 학습 모델 (pre-trained model)을 가져와 미세 조정으로 원하는 학습에 맞춤
- 사전 학습 모델 = 풀려는 문제와 비슷 + 많은 데이터로 이미 학습된 모델
 - 필요한 이유: 대량 데이터 확보도 어렵고, 학습에 시간 · 연산량이 많이 듦
 - 예: VGG, Inception, MobileNet
- 활용 방법 2가지: 특성 추출/ 미세 조정

★ 최종 정리표 ★

▼ 표 1-3 지도 학습, 비지도 학습, 강화 학습

구분	유형	알고리즘
지도 학습(supervised learning)	이미지 분류	• CNN • AlexNet • ResNet
	시계열 데이터 분석	• RNN • LSTM
비지도 학습(unsupervised learning)	군집 (clustering)	• 가우시안 혼합 모델(Gaussian Mixture Model, GMM) • 자기 조직화 지도(Self-Organizing Map, SOM)
	차원 축소	• 오토인코더(AutoEncoder) • 주성분 분석(PCA)
전이 학습(transfer learning)	전이 학습	• 버트(BERT) • MobileNetV2
강화 학습(reinforcement learning)	—	마르코프 결정 과정(MDP)

2장- 실습 환경 설정과 파이토치 기초

2.1 파이토치 개요

- 파이토치 (Pytorch): 2017년 초 공개된 딥러닝 프레임워크
 - Lua 언어로 개발된 Torch를 페이스북이 파이썬 버전으로 내놓은 것
 - Torch: 원래 Numpy처럼 과학 연산용 라이브러리로 공개 → 발전을 거듭해 딥러닝 프레임워크로
- 공식 튜토리얼의 정의 : “파이썬 기반 과학 연산 패키지로, 다음 두 집단을 대상”
 - NumPy를 대체하면서 GPU를 이용한 연산이 필요한 경우
 - 최대한의 유연성과 속도를 제공하는 딥러닝 연구 플랫폼이 필요한 경우



무엇보다 주목받는 이유 = 간결하고 빠른 구현성

2.1.1 파이토치 특징 및 장점



한마디 정의: GPU에서 텐서 조작 및 동적 신경망 구축이 가능한 프레임워크



Keyword: GPU / 텐서 / 동적 신경망

1. GPU (Graphics Processing Unit)-연산 속도 ↑

- 딥러닝은 기울기 계산에 미분을 쓰는데, GPU면 빠름
- 내부적으로 CUDA, cuDNN이라는 API를 통해 GPU 요요용
- 병렬 연산에서 CPU보다 훨씬 빠름 → 딥러닝 학습에 GPU는 필수

2. 텐서 (Tensor) - 파이토치의 데이터 형태

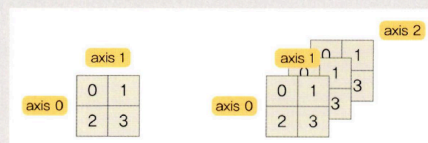
- 단일 데이터 형식으로 된 자료들의 다차원 행렬
- 변수 뒤에 `.cuda()` 만 붙이면 GPU로 연산

Note ≡ 벡터, 행렬, 텐서

인공지능(머신 러닝/딥러닝)에서 데이터는 벡터(vector)로 표현됩니다. 벡터는 [1.0, 1.1, 1.2]처럼 숫자들의 리스트로, 1차원 배열 형태입니다. 행렬(matrix)은 행과 열로 표현되는 2차원 배열 형태입니다. 이때 가로줄을 행(row)이라고 하며, 세로줄을 열(column)이라고 합니다. 마지막으로 텐서는 3차원 이상의 배열 형태입니다. 이를 정리하면 다음과 같습니다.

- 1차원 축(행)=axis 0=벡터
- 2차원 축(열)=axis 1=행렬
- 3차원 축(채널)=axis 2=텐서

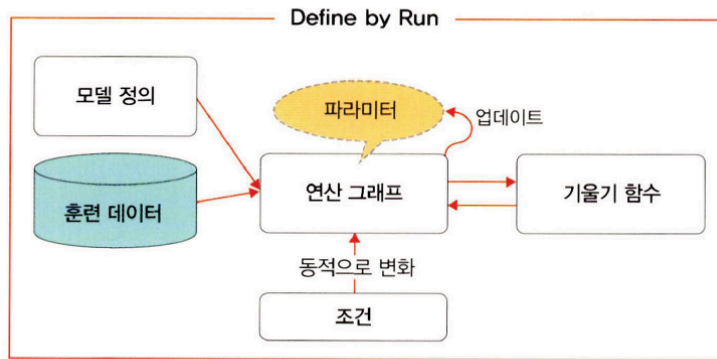
▼ 그림 2-2 벡터, 행렬, 텐서



3. 동적 신경망-훈련 반복 때마다 네트워크 변경 가능

- 학습 중 은닉층 추가·제거 등 네트워크 조작 가능
- Define By Run: 연산 그래프 정의와 동시에 값 초기화
 - 연산 그래프와 연산을 분리해서 생각할 필요 없음 → 코드 이해 쉬움

▼ 그림 2-1 파이토치 'Define by Run'

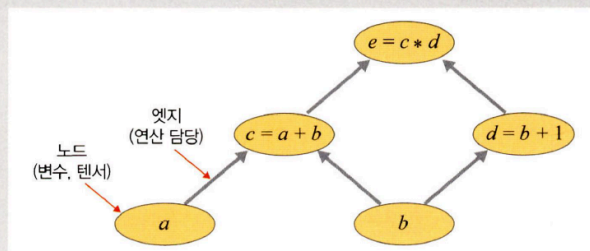


- 신경망 학습 = 손실 함수의 기울기를 가중치·바이어스 기반으로 계산 → 경사 하강법으로 업데이트. 이걸 연산 그래프로 효율적으로 수행

Note ≡ 연산 그래프

연산 그래프는 방향성이 있으며 변수(□ 텐서)를 의미하는 노드와 연산(□ 곱하기, 더하기)을 담당하는 엣지로 구성됩니다. 다음 그림과 같이 노드는 변수(a, b)를 가지고 있으며 각 계산을 통해 새로운 텐서(c, d, e)를 구성할 수 있습니다.

▼ 그림 2-3 파이토치 연산 그래프



신경망은 연산 그래프를 이용하여 계산을 수행합니다. 즉, 네트워크가 학습될 때 손실 함수의 기울기가 가중치와 바이어스를 기반으로 계산되며, 이후 경사 하강법을 사용하여 가중치가 업데이트됩니다. 이때 연산 그래프를 이용하여 이 과정이 효과적으로 수행됩니다.

PyTorch의 장점

- 단순함(효율적인 계산)
 - 파이썬 환경과 쉽게 통합 가능
 - 디버깅이 직관적이고 간결
- 성능 (낮은 CPU 활용)
 - 텐서플로와 비교했을 때 모델 훈련을 위한 CPU 사용률이 낮음
 - 학습 및 추론 속도가 빠르고 다루기 쉬움

- 직관적인 인터페이스

- 텐서플로처럼 잦은 API 변경이 쉬움

2.1.2 파이토치의 아키텍처

이해하기 쉬운 비유: 카페 구조로 바라보기

계층	하는 일	비유
파이토치 API (Python)	사용자가 코드로 만지는 부분. 계산은 안 하고 아래로 넘기기만 함	키오스크·메뉴판
파이토치 엔진 (C++)	다차원 텐서 연산 + 자동 미분을 실제로 처리	바리스타
연산 처리 (C/CUDA)	CPU·GPU에서 진짜 숫자를 계산	에스프레소 머신

💡 핵심: 위로 갈수록 사람이 쓰기 편하고, 아래로 갈수록 빠르다. 파이썬으로 쓰는데도 빠른 이유가 이 구조 — 겉데기만 파이썬, 알맹이는 C++/CUDA.

① 파이토치 API — 사용자용 패키지 5종

패키지	역할	기억할 포인트
torch	GPU 지원 텐서 패키지	다차원 텐서 + 수학 연산의 기본
torch.autograd	자동 미분	TF·Caffe·CNTK와 가장 차별화되는 지점. 연산 그래프가 실행 시점에 즉시 생성(동적 그래프)되어 망을 조금 바꿔도 처음부터 다시 안 짜도 됨
torch.nn	신경망 구축·훈련	합성곱, 순환, 정규화 등이 이미 들어있음
torch.multiprocessing	파이썬 멀티프로세싱	프로세스 간 텐서 메모리 공유 → 같은 데이터에 여러 프로세스가 접근
torch.utils	유틸리티	data.DataLoader(데이터 공급), bottleneck(병목 디버깅), checkpoint(모델 일부 검사)

② 파이토치 엔진 — C++ 4형제

- **Autograd C++** : 가중치·바이어스 업데이트에 필요한 미분을 자동 계산
- **Aten C++** : C++ 텐서 라이브러리 (실제 텐서 자료구조)
- **JIT C++** : 계산 최적화용 Just-In-Time 컴파일러 (바이트코드 → CPU가 바로 실행할 명령어)
- **Python API** : 위 C++ 라이브러리를 감싸서(auto wrapping) 파이썬 문법으로 내주는 층

흐름: 파이썬 프로그램 → torch / torch.nn → autograd 포함 C++ 라이브러리 → Aten(텐서) → 백엔드(CUDA, cuDNN)

③ 연산 처리 — C / CUDA

- TH(CPU) / THC(GPU) : 텐서 연산
- THNN(CPU) / THCUNN(GPU) : 신경망 연산
- 상위 API가 시킨 계산은 사실상 전부 여기서 수행됨

Note. 텐서를 메모리에 저장하기 (스토리지·오프셋·스트라이드)

전제: 텐서가 몇 차원이든 메모리에는 반드시 1차원 배열로 눕혀서 저장된다. 이 눕힌 1차원 배열이 **스토리지(storage)**.

그럼 2차원인지 3차원인지는 어떻게 알까? → 설명서 세 개가 따로 붙어 있다.

용어	의미	비유
형태(shape)	(행, 열)	몇 칸씩 접어 읽을지
오프셋(offset)	첫 요소가 스토리지의 몇 번째 인덱스에 있는지	어디서부터 읽기 시작할지
스트라이드(stride)	각 차원에서 다음 요소로 가려면 몇 칸 건너뛰는지	몇 칸씩 점프할지

왜 필요한가 — 전치 행렬 예시

- $A(2 \times 3)$ 와 $A^T(3 \times 2)$ 는 **형태는 다른데 스토리지 값 자체는 같다**.
- 그래서 둘을 구분하는 용도로 오프셋·스트라이드를 쓴다.

텐서	shape	stride	읽는 법
A	(2, 3)	(3, 1)	+1 → 다음 열 / +3 → 다음 행
A^T	(3, 2)	(2, 1)	+1 → 다음 열 / +2 → 다음 행

💡 결론 한 줄: 오프셋과 스트라이드는 데이터 자체가 아니라 "그 데이터를 어떤 행렬/텐서로 볼지"를 구분하는 장치다. 같은 서랍장, 설명서만 바꾸는 것.

⚠ 실제 PyTorch에서 A.T는 값을 새로 배치하지 않고 stride만 (1, 3)으로 바꿔 같은 스토리지를 공유한다. 나중에 .contiguous(), view() vs reshape()의 차이가 여기서 나온다.

2.2 파이토치 기본 문법

2.2.1 텐서 다루기

텐서 생성 및 변환

텐서 = 파이토치의 가장 기본 데이터 구조. **넘파이 ndarray와 거의 같은데 GPU 연산이 된다는 게 차이.**

목적	코드
2차원 텐서 생성	<code>torch.tensor([[1,2],[3,4]])</code>
GPU에 생성	<code>torch.tensor([[1,2],[3,4]], device="cuda:0")</code>
자료형 지정	<code>torch.tensor([[1,2],[3,4]], dtype=torch.float64)</code>
텐서 → ndarray	<code>temp.numpy()</code>
GPU 텐서 → ndarray	<code>temp.to("cpu").numpy()</code>

⚠ GPU에 있는 텐서는 곧바로 `numpy()`가 안 된다. 넘파이는 CPU 메모리에만 사는 라이브러리라서 `.to("cpu")`로 옮긴 뒤에야 변환된다.

주요 자료형

타입	의미
<code>torch.FloatTensor</code>	32비트 부동 소수점
<code>torch.DoubleTensor</code>	64비트 부동 소수점
<code>torch.LongTensor</code>	64비트 부호 있는 정수

텐서의 인덱스 조작

넘파이 배열과 동일하게 인덱스·슬라이스 사용 가능.

- `temp[0]`, `temp[1]` → 인덱스로 접근
- `temp[-1]` → 뒤에서부터 세는 인덱스 (맨 마지막)
- `temp[2:5]` → 2번부터 4번까지 (끝 번호는 미포함)
- `temp[4:-1]` → 4번부터 마지막 직전까지

텐서 연산 및 차원 조작

- 넘파이처럼 사칙연산 가능, GPU 쓰면 더 빠름
- ⚠ 타입이 다르면 연산 불가. `FloatTensor`와 `DoubleTensor`를 더하면 오류

- 길이가 같은 벡터끼리 원소별 연산

차원 조작 4형제

함수	역할	메모
view	차원(형태) 변경	넘파이 reshape과 유사. 가장 대표적
stack	텐서 결합	새 차원을 만들며 쌓음
cat	텐서 병합	서로 다른 길이의 텐서를 하나로
t / transpose	차원 교환	전치 외에 차원 순서 변경에도 사용

view의 -1 규칙

- 1은 "다른 차원 보고 알아서 계산해"라는 뜻. **원소 개수는 그대로 유지**되어야 하므로 답이 하나로 정해진다.

코드	결과 형태	이유
temp.view(4, 1)	(4, 1)	2×2=4개를 4행 1열로
temp.view(-1)	(4,)	1차원 벡터로 펴기
temp.view(1, -1)	(1, 4)	원소 4개 유지 → ?=4
temp.view(-1, 1)	(4, 1)	원소 4개 유지 → ?=4

💡 텐서의 차원 문제는 신경망에서 계속 마주치니까 view의 -1만큼은 손에 익혀둘 것.

2.2.2 데이터 준비

데이터 호출 방법은 두 갈래: **판다스로 직접 불러오기** vs **파이토치가 제공하는 데이터 사용**.

데이터 종류	처리 흐름
이미지	분산된 파일에서 읽기 → 전처리 → 배치 단위로 분할
텍스트	임베딩(자연어 → 숫자 벡터) → 길이가 다른 시퀀스를 배치 단위로 분할

① 단순하게 파일을 불러와서 사용

판다스로 JSON, CSV 등을 읽는 방식. **데이터가 복잡하지 않을 때** 단순하고 유용하다.

핵심 변환 한 줄:

```
torch.from_numpy(data['x'].values).unsqueeze(dim=1).float()
```

단계	하는 일
.values	판다스 칼럼 → 넘파이 배열
torch.from_numpy()	넘파이 배열 → 텐서

단계	하는 일
<code>.unsqueeze(dim=1)</code>	차원 추가 (예: $(N,)$ $\rightarrow$ $(N, 1)$)
<code>.float()</code>	자료형을 float으로

② 커스텀 데이터셋을 만들어서 사용

문제: 딥러닝은 대량 데이터를 쓰는데, 한 번에 다 메모리에 올리면 시간·비용이 비효율적이다.

해결: 한 번에 다 부르지 않고 **조금씩 나누어 불러오는 방식** = 커스텀 데이터셋.

`class CustomDataset(torch.utils.data.Dataset)` 를 상속하고 메서드 3개를 구현한다.

메서드	역할
<code>init(self)</code>	필요한 변수 선언 + 데이터셋 전처리
<code>len(self)</code>	데이터셋의 길이(총 샘플 수) 반환
<code>getitem(self, index)</code>	index번째 데이터 반환. 반환값은 반드시 텐서 형태

③ DataLoader

`dataset = DataLoader(tensor_dataset, batch_size=4, shuffle=True)`

- 학습에 쓸 **데이터 전체를 보관했다가**, 학습할 때 배치 크기만큼 꺼내 준다
- ⚠ 주의: 데이터를 미리 잘라 놓는 게 아니다. 내부 반복자(iterator)에 들어있는 인덱스를 이용해 그때그때 배치 크기만큼 반환한다
- 흐름: 전체 데이터셋 $\rightarrow$ 랜덤으로 섞기(shuffle) $\rightarrow$ 반복(iterable) $\rightarrow$ 배치 1, 배치 2, ...

💡 Dataset은 "한 개를 어떻게 꺼내는지"를 정의하고, DataLoader는 "여러 개를 어떻게 묶어서 줄지"를 담당한다. 역할이 딱 나뉜다.

2.2.3 모델 정의

파이토치에서 모델 = `nn.Module`을 상속한 클래스
그런데 계층·모듈·모델은 "종류"가 아니라 "범위" 차이

계층 · 모듈 · 모델

- 계층(layer): 제일 작은 단위. 블록 한 개 — `nn.Linear`, `nn.Conv2d`, `nn.ReLU`

- 모듈(module): 계층 묶음. 블록 몇 개 붙인 덩어리 — `nn.Sequential(Conv2d, ReLU, MaxPool2d)`. 모듈이 모여 새 모듈이 될 수도 있음
- 모델(model): 제일 바깥 완성품. 최종적으로 원하는 네트워크 — `MLP()`. 모듈 하나가 곧 모델일 수도 있음

헷갈리는 이유: 파이토치에선 셋 다 `nn.Module`

`nn.Linear`도, `nn.Sequential`도, 내가 만든 `MLP`도 전부 `Module`

→ 물건에 붙은 이름표가 아니라 "부품으로 보느냐, 완성품으로 보느냐"에 따라 바뀌는 호칭

→ `nn.Linear(1,1)` 혼자 쓰면 모델, `MLP` 안에 들어가면 계층

모델을 정의하는 방법

방법	핵심	언제 쓰나	형태
단순 신경망	계층 하나를 그대로 모델로 씬. 클래스 · <code>forward</code> 없음. 연산이 "입력 → 계층 → 출력" 하나 순서를 적을 필요가 없음	계층 1개 / 관계가 선형 / 데이터 적음 / 해석 필요 / 베이스라인	<pre>model = nn.Linear(1, 1)</pre>
<code>nn.Module</code> 상속	<code>__init__</code> = 재료 준비(어떤 계층 쓸지) / <code>forward</code> = 조리 순서(어떤 순서로 통과시킬지). 클래스가 이 둘을 담는 그릇	계층 여러 개 + 연산 순서를 직접 정해야 할 때 (분기, 반복, skip connection, if문)	<pre>class MLP(nn.Module)</pre>
<code>nn.Sequential</code>	여러 계층을 한 묶음으로. <code>forward</code> 가 짧아지고 순서대로 자동 실행. 클래스 없이도 모델이 됨	계층이 여러 개지만 일렬로 통과만 시킬 때. 계층이 복잡할수록 효과 큼	<pre>nn.Sequential(Conv2d, ReLU, MaxPool2d)</pre>
함수로 정의	계층들을 변수에 담아 <code>nn.Sequential</code> 로 묶어 반환하는 함수. 동작은 <code>Sequential</code> 과 동일	같은 구조의 모델을 크기만 바꿔 여러 번 만들 때 (인자로 크기 조절). 복잡한 모델엔 부적합 → <code>nn.Module</code> 상속이 편함	<pre>def MLP(...): return nn.Sequential(...)</pre>

함수로 신경망을 정의하는 방법

- Sequential과 동일한데, 계층을 함수 안 변수(`hidden` , `activation` , `output`)에 저장해 두고 마지막에 `nn.Sequential` 로 묶어 `return`
- 장점: 변수에 저장해 둔 계층을 재사용할 수 있음. 함수 인자로 `in_features` , `hidden_features` , `out_features` 를 받으니까 `MLP(1, 20, 1)` , `MLP(10, 50, 3)` 처럼 크기만 바꿔 같은 구조를 여러 번 찍어낼 수 있음
- 단점: 모델이 복잡해지면 함수가 길고 지저분해짐 → 복잡한 모델은 `nn.Module` 상속이 편함
- ReLU·Softmax·Sigmoid 같은 활성화 함수는 모델을 정의할 때 지정

클래스 방식과의 차이

- 클래스(`nn.Module` 상속): `forward`를 직접 써서 연산 순서·분기를 내가 정함
- 함수: `forward`를 안 씀. `Sequential`이 순서를 정함 → 일렬 구조에만 가능
- 즉 "함수 방식 = Sequential을 크기 조절 가능한 공장으로 만든 것"

갈림길은 하나: 연산 순서를 내가 적어야 하는가?

- 계층 하나 → 적을 게 없음 → 단순 신경망
- 여러 개인데 일렬 → `nn.Sequential`
- 여러 개인데 일렬이 아님(분기·합치기·조건) → `nn.Module` 상속 + `forward` 직접 작성

심층 신경망이 있어도 단순 신경망이 나은 경우

- 데이터가 실제로 선형일 때 — 키 → 몸무게, 광고비 → 매출. $y = wx + b$ 로 충분. 층을 쌓으면 정확도는 안 오르고 과적합 위험만 증가
- 데이터가 적을 때 — 심층망은 파라미터가 많아 적은 데이터를 외워버림(과적합). 수백 개 규모면 선형 모델이 이기는 경우가 많음
- 해석이 필요할 때 — `nn.Linear` 하나면 가중치 w 를 보고 "이 변수가 결과를 이만큼 밀어올린다"를 바로 말할 수 있음. 심층망은 불가 → 의료·규제 환경에서 단순 모델이 채택되는 이유
- 베이스라인 — 복잡한 모델 전에 단순 모델을 먼저 돌려 기준점을 만듦. 이게 없으면 층을 쌓은 게 의미 있는지 알 수 없음

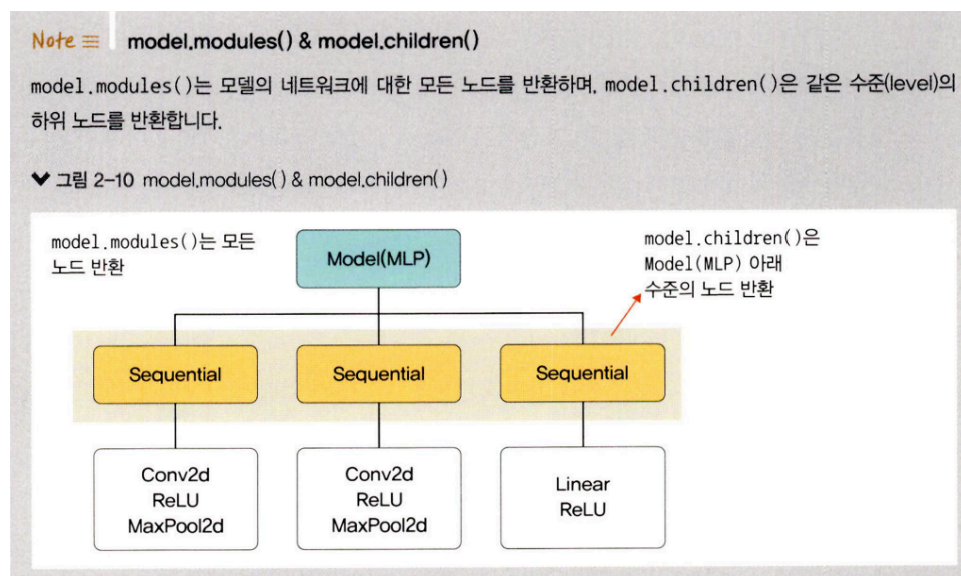
- 학습 파이프라인 자체를 익힐 때 — 손실·옵티마이저·학습 루프가 목적이면 모델은 제일 단순한 게 방해가 안 됨 (교재 2장이 이 경우)

"심층"은 항상 상위 호환이 아님

- 모델의 깊이는 데이터의 복잡도에 맞춰 고르는 것
- 데이터가 단순한데 모델만 깊으면 오히려 손해

model.children() vs model.modules()

- `children()` : 바로 아래 자식만 → `[Sequential, Sequential, Sequential]`
- `modules()` : 나 자신(MLP)부터 맨 아래 Conv2d·ReLU까지 전부
- 자식만 볼 거면 `children`, 트리 전체 훑을 거면 `modules`



2.2.4 모델의 파라미터 정의

모델을 학습하기 전에 필요한 것들을 미리 정해두는 단계

손실 함수(채점 기준) → 옵티마이저(고치는 방법) → 학습률 스케줄러(고치는 폭 조절)
→ 지표(성적표)

손실 함수 (loss function)

- 학습하는 동안 출력과 실제 값(정답) 사이의 오차를 측정. 즉 $w \cdot x + b$ 로 계산한 값과 실제 값 y 의 차이를 구해 모델의 정확성을 측정
- 오차가 클수록 손실 함수 값이 크고, 작을수록 작아짐 → 이 값을 최소로 만드는 가중치 w 와 바이어스 b 를 찾는 것이 학습의 목표

손실 함수	언제	예
<code>BCELoss</code>	이진 분류	양성/음성, 스팸/정상
<code>CrossEntropyLoss</code>	다중 클래스 분류	숫자 0~9, 개/고양이/새
<code>MSELoss</code>	회귀	집값, 온도 예측

옵티마이저 (optimizer)

- 데이터와 손실 함수를 바탕으로 모델의 업데이트 방법을 결정 (오차를 보고 w , b 를 어느 방향으로 얼마나 고칠지)
- `step()` 메서드로 전달받은 파라미터를 업데이트
- 모델의 파라미터별로 다른 기준(예: 학습률)을 적용할 수 있음
- `torch.optim.Optimizer(params, defaults)` 는 모든 옵티마이저의 기본이 되는 클래스
- `zero_grad()` 메서드는 옵티마이저에 사용된 파라미터들의 기울기(gradient)를 0으로 만듦
- `torch.optim.lr_scheduler` 는 에포크에 따라 학습률을 조절
- 종류 (4장에서 자세히): Adadelta, Adagrad, Adam, SparseAdam, Adamax / ASGD, LBFGS / RMSProp, Rprop, SGD

`zero_grad()`가 왜 필요한가

- 파이토치는 기울기를 계산할 때 이전 값에 **누적함** (RNN에서는 필요한 방식)
- 일반 모델은 누적이 필요 없으니 매 반복 전에 0으로 초기화해야 함
- 안 하면 이전 배치의 기울기가 섞여서 엉뚱한 방향으로 업데이트됨

학습률 스케줄러 (learning rate scheduler)

- 미리 지정한 횟수의 에포크를 지날 때마다 학습률을 감소(decay)시킴
- 학습 초기에는 빠르게 학습하다가 전역 최소점 근처에 다다르면 학습률을 줄여 최적점을 정밀하게 찾아감 (멀리서는 큰 걸음, 가까워지면 작은 걸음)

스케줄러	동작
<code>LambdaLR</code>	람다 함수의 결과를 학습률로 설정
<code>StepLR</code>	특정 단계(step)마다 학습률을 감마(gamma) 비율만큼 감소
<code>MultiStepLR</code>	StepLR과 비슷하지만 지정된 에포크에만 감마 비율로 감소
<code>ExponentialLR</code>	에포크마다 이전 학습률에 감마를 곱함

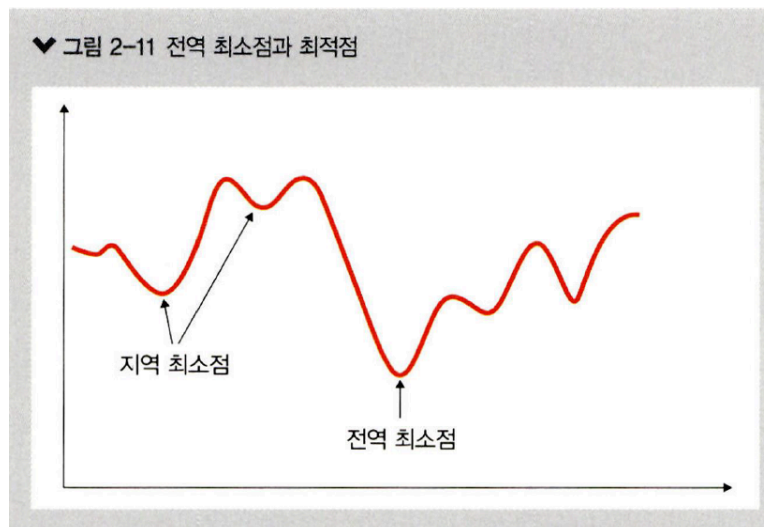
스케줄러	동작
<code>CosineAnnealingLR</code>	학습률을 코사인 함수 형태로 변화 → 커지기도 작아지기도 함
<code>ReduceLR0nPlateau</code>	학습이 잘되고 있는지 아닌지에 따라 동적으로 학습률 변화

지표 (metrics)

- 훈련과 테스트 단계를 모니터링 (정확도, 손실 값 등)

전역 최소점과 지역 최소점

- 전역 최소점(global minimum): 오차가 가장 작을 때의 값 → 우리가 최종적으로 찾고자 하는 최적점
- 지역 최소점(local minimum): 전역 최소점을 찾아가는 과정에서 만나는 홀(hole). 옵티마이저가 여기서 학습을 멈추면 최솟값을 갖는 오차를 찾을 수 없음
- 산에서 내려올 때 작은 웅덩이(지역)에 갇히면 진짜 골짜기(전역)에 못 감 → 옵티마이저 스케줄러가 이걸 빠져나오게 돕는 장치



예시 코드 흐름

- `criterion = torch.nn.MSELoss()` → 손실 함수
- `optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)` → 옵티마이저에 모델의 파라미터(w, b)를 넘겨줌
- `scheduler = LambdaLR(optimizer, lr_lambda=lambda epoch: 0.95 ** epoch)` → 에포크마다 학습률을 0.95배

- 학습 루프: 에포크 반복 → 배치 반복 → `zero_grad()` → `backward()` → `optimizer.step()` → `scheduler.step()`

2.2.5 모델 훈련

- 앞서 만들어 둔 데이터로 모델을 학습시킴. 학습 = $y = wx + b$ 에서 w와 b의 적절한 값을 찾는 것
- w, b에 임의의 값을 넣고 시작 → 오차가 줄어들어 전역 최소점에 이를 때까지 파라미터를 계속 수정
- 가장 먼저 필요한 절차: `optimizer.zero_grad()` 로 기울기 초기화
- 파이토치는 기울기 계산에 `loss.backward()` 를 쓰는데, 새 기울기가 이전 기울기에 누적됨 → RNN엔 효과적이지만 일반 모델엔 불필요 → 입력을 모델에 넣기 전에 `zero_grad()` 로 미분 값을 초기화해야 함

▼ 그림 2-12 파이토치 학습 절차

딥러닝 학습 절차	파이토치 학습 절차
모델, 손실 함수, 옵티마이저 정의	모델, 손실 함수, 옵티마이저 정의
전방향 학습(입력 → 출력 계산)	<code>optimizer.zero_grad()</code> : 전방향 학습, 기울기 초기화
손실 함수로 출력과 정답의 차이(오차) 계산	<code>output = model(input)</code> : 출력 계산
역전파 학습(기울기 계산)	<code>loss = loss_fn(output, target)</code> : 오차 계산
기울기 업데이트	<code>loss.backward()</code> : 역전파 학습
	<code>optimizer.step()</code> : 기울기 업데이트

↑
모델 학습 과정

학습 루프 한 바퀴 = 지우고 → 예측하고 → 채점하고 → 어디를 고칠지 계산하고 → 고친다
이걸 에포크 수만큼 반복

2.2.6 모델 평가

- 테스트 데이터셋으로 모델을 평가. 함수 방식과 모듈 방식 두 가지
- 사전 설치: `pip install torchmetrics`

방식	코드	특징
함수	<code>torchmetrics.functional.accuracy(preds, target)</code>	한 번 호출 → 그 데이터의 정확도 바로 반환

방식	코드	특징
모듈	<pre>metric = torchmetrics.Accuracy() → 배치마다 metric(preds, target) → 마지막에 metric.compute()</pre>	배치별 정확도를 누적해 두었다가 compute()로 전체 정확도 산출

- 대안: 사이킷런 `metrics` 모듈의 `confusion_matrix`, `accuracy_score`, `classification_report`로도 정확도 계산 가능

함수 방식 = 시험 한 번 채점, 모듈 방식 = 시험 여러 번 채점하고 마지막에 평균 내기

2.2.7 훈련 과정 모니터링

- 학습 중 각 파라미터가 어떻게 변하는지 눈으로 보기 어려움 → **텐서보드**로 시각화. 성능 추적·평가 용도로도 사용
- 사용 순서: 1. 텐서보드 설정(set up) → 2. 텐서보드에 기록(write) → 3. 텐서보드로 모델 구조 확인
- 사전 설치: `pip install tensorboard`
- 핵심 코드
 - `writer = SummaryWriter("../chap02/tensorboard")` → 모니터링 값이 저장될 위치
 - `writer.add_scalar("Loss", loss, epoch)` → 스칼라 값(오차) 기록
 - `writer.close()` → SummaryWriter가 더 이상 필요 없으면 호출
- 실행: 터미널에서 `tensorboard --logdir=../chap02/tensorboard --port=6006` → 브라우저에서 `http://localhost:6006`

model.train() & model.eval()

	model.train()	model.eval()
언제	훈련 데이터셋	검증·테스트 데이터셋
의미	지금부터 모델 훈련이 진행됨	모델을 평가 → 모든 노드를 사용
드롭아웃	활성화 (True)	비활성화 (False)

- 둘 다 선언해야 모델의 정확도를 높일 수 있음
- `model.eval()` 은 `with torch.no_grad():` 와 함께 사용

with torch.no_grad()를 쓰는 이유

- 파이토치는 모든 연산과 기울기 값을 저장함

- 검증·테스트에서는 역전파가 필요 없으므로 기울기를 저장하지 않게 함
- 기울기 저장·기록에 드는 메모리와 연산 시간을 줄일 수 있음

2.4 파이토치 코드 맛보기

단계	하는 일	핵심 코드
① 데이터 호출	csv → DataFrame	<code>pd.read_csv</code> , <code>head()</code>
② 전처리	단어 → 숫자 → 텐서	<code>astype('category')</code> , <code>cat.codes</code> , <code>np.stack</code> , <code>torch.tensor</code>
③ 레이블 처리	문자 레이블 → 0/1 → 텐서	<code>pd.get_dummies</code> , <code>flatten()</code>
④ 임베딩 크기 정의	(고유값 수, 차원) 쌍	<code>min(50, (col_size+1)//2)</code>
⑤ 데이터 분리	훈련 80% / 테스트 20%	슬라이싱
⑥ 모델 정의	<code>nn.Module</code> 상속 클래스	<code>__init__</code> , <code>forward</code>
⑦ 손실·옵티마이저	분류 → 크로스 엔트로피, Adam	<code>nn.CrossEntropyLoss</code> , <code>torch.optim.Adam</code>
⑧ 학습	500 에포크 반복	<code>zero_grad</code> → <code>backward</code> → <code>step</code>
⑨ 평가	테스트 손실, argmax, 지표	<code>torch.no_grad</code> , <code>np.argmax</code> , <code>sklearn.metrics</code>

비유: 요리 레시피- 재료 손질(전처리) → 재료 계량(텐서/분리) → 조리 도구 준비(모델·손실·옵티마이저) → 조리(학습) → 시식(평가).

1. 데이터셋 (car_evaluation.csv)

- 특성(칼럼) 7개: `price` (가격), `maint` (유지 비용), `doors` (문 개수), `persons` (수용 인원), `lug_capacity` (수하물 용량), `safety` (안전성), `output` (차 상태)
- `output` 값: `unacc` (허용 불가) / `acc` (허용 가능) / `good` (양호) / `vgood` (매우 좋음)
- 총 1728행
- 분포(그림 2-26 파이 차트): `unacc` 약 70%, `acc` 약 22%, `good`·`vgood` 각 약 4%
→ 대부분 허용 불가, 양호한 차 비율은 매우 낮음(클래스 불균형)
- `dataset.head()` : 처음 n줄(기본 5) 출력, 인덱스는 0부터 시작
- `dataset.tail()` : 마지막 n줄 출력

2. 전처리 — 단어를 숫자로

딥러닝은 통계 알고리즘 기반 → 단어를 **숫자(텐서)** 로 변환!! 예제의 칼럼은 모두 **범주형 데이터**(예: 성별 여/남)다.

- 변환 절차:

범주형 데이터 → **dataset[category]** → **넘파이 배열(NumPy array)** → **텐서(Tensor)**

- `astype('category')` : 범주 특성을 갖는 데이터를 범주형 타입으로 변환
- `cat.codes` : 범주형 데이터(단어)를 숫자(넘파이 배열)로 변환
 - ⚠ 어떤 클래스가 어떤 숫자로 매핑됐는지 확인하기 어려움 → 주의해서 사용
- `np.stack(..., 1)` : 두 개 이상의 넘파이 객체를 합침 (여기서는 6개 칼럼을 열 방향으로 붙여 (1728, 6) 배열 생성)
- `torch.tensor(arr, dtype=torch.int64)` : 넘파이 배열 → 텐서

비유: `cat.codes` 는 단어에 번호표를 붙이는 것. 번호표만 보면 원래 단어가 뭔지 알기 어렵다는 게 단점.

Note: np.stack vs np.concatenate

	<code>np.concatenate</code>	<code>np.stack</code>
차원	유지 (2D + 2D → 2D)	새 축 추가 (2D + 2D → 3D)
동작	선택한 축(axis)을 기준으로 두 배열을 연결	배열들을 새로운 축 으로 합침
조건	합치는 축 외 크기만 맞으면 됨	두 배열의 차원(shape) 이 반드시 동일해야 함
예	<code>a(2,2)+b(2,2), axis=0 → (4,2)</code>	<code>a(2,2)+b(2,2), axis=0 → (2,2,2)</code>
다른 shape	<code>a(2,2)+c(3,2), axis=0 → (5,2) OK</code>	<code>a(2,2)+c(3,2) → ValueError: all input arrays must have the same shape</code>

3. 레이블(outputs) 처리

- `pd.get_dummies()` : **가변수(dummy variable)** 로 만들어 주는 함수 = 문자를 숫자 (0, 1)로 바꿔 줌

- 예: gender/weight/nation → `weight`, `gender_female`, `gender_male`, `nation_Australia`, `nation_Japan`, `nation_Korea`
- 원래 숫자였던 `weight` 는 변화 없고, 문자인 성별·국적만 0/1로 변환
- `torch.tensor(outputs).flatten()` : 1차원 텐서로 변환
- 결과 shape: `categorical_data` → `torch.Size([1728, 6])`, `outputs` → `torch.Size([6912])` (1728×4)

Note: ravel(), reshape(-1), flatten()

- 셋 다 텐서의 차원을 바꿀 때 사용, 2차원 → 1차원으로 펼침
- `[[1,2],[3,4]]` → 셋 모두 `[1 2 3 4]`

비유: get_dummies는 "국적" 한 칸을 "일본인가? 한국인가? 호주인가?" 예/아니오 (1/0) 질문 여러 칸으로 쪼개는 것.

4. 워드 임베딩 & 임베딩 크기

- 워드 임베딩: 유사한 단어끼리 유사하게 인코딩되도록 표현하는 방법
- 높은 차원의 임베딩일수록 단어 간 세부 관계를 잘 파악 → 단일 숫자로 변환된 넘파이 배열을 **N차원**으로 변경해 사용
- 임베딩 크기 규칙: 정확한 규칙은 없지만 **칼럼의 고유 값 수를 2로 나눈 값**을 많이 사용
 - 예: price 고유값 4개 → $4/2 = 2$
- 코드: `(col_size, min(50, (col_size+1)//2))` → 결과 `[(4,2), (4,2), (4,2), (3,2), (3,2), (3,2)]` = (고유 값 수, 차원 크기)

5. 데이터셋 분리

- 전체 1728개 중 **20%를 테스트 용도** → `test_records = int(1728 * .2) = 345`
- 훈련 1383 / 테스트 345 (`len()` 으로 확인)

6. 모델 네트워크 (코드 2-10)

- ① 클래스 형태의 모델은 `nn.Module` 을 상속받는다
- ② `__init__()` : 모델에서 사용할 파라미터와 신경망을 초기화, 객체 생성 시 자동 호출

매개변수	의미
① <code>self</code>	첫 번째 파라미터는 반드시 self, 자기 자신

매개변수	의미
② <code>embedding_size</code>	범주형 칼럼의 임베딩 크기
③ <code>output_size</code>	출력층의 크기
④ <code>layers</code>	모든 계층에 대한 목록
⑤ <code>p</code>	드롭아웃 (기본값 0.5)

- ③ `super().__init__()` : 부모 클래스에 접근할 때 사용, **super는 self를 사용하지 않음**에 주의
- ④ for 문으로 각 계층을 `all_layers` 목록에 추가:
 - `Linear` : 선형 계층, 입력에 선형 변환 $\rightarrow y = Wx + b$ (y: 출력, W: 가중치, x: 입력, b: 바이어스) = 입력과 가중치를 곱한 후 바이어스를 더한 결과
 - `ReLU` : 활성화 함수
 - `BatchNorm1d` : 배치 정규화 — 신경망 안에서 데이터의 평균과 분산을 조정 (보통 평균 0, 분산 1)
 - `Dropout` : 과적합 방지
- `nn.Sequential(*all_layers)` : 모든 계층이 **순차적으로 실행**되도록 목록을 전달
- ⑤ `forward()` : 학습 데이터를 입력받아 연산, **모델 객체를 데이터와 함께 호출하면 자동 실행**
 - `torch.cat(embeddings, 1)` : 넘파이의 concatenate와 같지만 대상이 텐서

비유: `__init__` 은 공장 설비 설치(레이어들을 만들어 두기), `forward` 는 실제로 컨베이어 벨트 돌리기(데이터 흘려보내기). `nn.Sequential` 은 설비들을 한 줄로 이어 놓은 컨베이어 벨트.

- 객체 생성: `Model(categorical_embedding_sizes, 4, [200,100,50], p=0.4)` $\rightarrow$ 은닉층 [200,100,50], 출력 4
 - 입력 크기 12 = 임베딩 차원 합(2×6)
 - 구조: Embedding 6개 $\rightarrow$ Dropout(0.4) $\rightarrow$ [Linear $\rightarrow$ ReLU $\rightarrow$ BatchNorm1d $\rightarrow$ Dropout] $\times$ 3 $\rightarrow$ Linear(50, 4)

7. 손실 함수 · 옵티마이저 · 디바이스

- 분류 문제 $\rightarrow$ **크로스 엔트로피(cross entropy) 손실 함수**, 옵티마이저는 **아담(Adam)** (`lr=0.001`)

- 파이토치는 GPU에 최적화 → GPU 없으면 CPU 쓰도록 지정:

`torch.cuda.is_available()` 로 분기

8. 모델 학습 (코드 2-14)

- `epochs = 500`, 매 반복마다 손실 계산 → `aggregated_losses` 에 추가
- 한 스텝의 3박자:
 1. `optimizer.zero_grad()` : 기울기 초기화
 2. `single_loss.backward()` : 가중치 업데이트를 위해 손실 함수의 `backward()` 호출
 3. `optimizer.step()` : 옵티마이저의 `step()`으로 기울기 업데이트
- 25 에포크마다 오차 출력 → loss가 1.64 → 0.57로 감소

비유: `zero_grad`는 칠판 지우기, `backward`는 "어느 방향으로 고쳐야 하는지" 계산, `step`은 실제로 한 걸음 이동. 칠판을 안 지우면 이전 방향이 누적된다.

9. 평가

- `torch.no_grad()` 안에서 테스트 예측 → `Loss: 0.555` ≈ 훈련 손실(0.573) → **과적합 없음**으로 판단
- 출력은 예측당 4개 값(`output_size=4`) → 실제 값이 0이면 인덱스 0의 값이 가장 커야 함
- `np.argmax(y_val, axis=1)` : 가장 큰 값이 아닌 인덱스를 찾음 → `tensor([0,0,0,0,0])`
- `sklearn.metrics` : `confusion_matrix`, `classification_report`, `accuracy_score` → 정확도 약 75%
 - 파라미터를 무작위로 선택한 것 감안하면 나쁘지 않음. 훈련/테스트 분할, 은닉층 개수·크기 등을 바꿔 보는 것이 학습에 도움

10. 분류 성능 평가 지표

용어	예측	실제	별칭
True Positive (TP)	1	1	
True Negative (TN)	0	0	
False Positive (FP)	1	0	Type I 오류
False Negative (FN)	0	1	Type II 오류

지표	의미	공식	언제
정확도 (accuracy)	전체 예측 중 정답 비율 (positive/negative 무관)	$(TP+TN) / (TP+TN+FP+FN)$	전반적 성능
재현율 (recall)	실제 정답이 1일 때 모델도 1로 예측한 비율	$TP / (TP+FN)$	데이터가 1일 확률이 적을 때
정밀도 (precision)	모델이 1로 예측한 것 중 실제 1인 비율	$TP / (TP+FP)$	오탐(FP)이 치명적일 때
F1-스코어	정밀도·재현율의 조화 평균	$2 \times (P \times R) / (P+R)$	둘의 트레이드오프 해결

- 정밀도와 재현율은 **트레이드오프** 관계: 정밀도↑ → 재현율↓, 재현율↑ → 정밀도↓

비유: 재현율은 "진짜 환자를 얼마나 놓치지 않고 잡았나"(그물의 촘촘함), 정밀도는 "환자라고 한 사람 중 진짜 환자가 얼마나 되나"(그물에 잡힌 것 중 물고기 비율). 그물을 크게 치면 재현율↑ 정밀도↓.